Ayudantía 3

SISTEMAS DISTRIBUIDOS

PROFESOR: NICOLAS HIDALGO

AYUDANTE CÁTEDRA: VICENTE DÍAZ, JOAQUÍN VILLEGAS

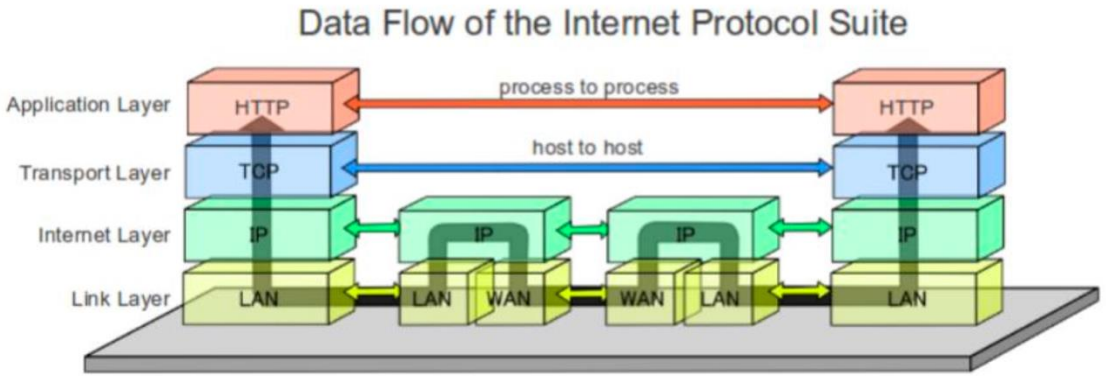
AYUDANTE CORRECTOR: BENJAMÍN ACEITUNO, ISIDORA GONZALEZ, NATALIA ORTEGA

Contacto

Benjamín Aceituno
benjamin.aceituno@mail.udp.cl
Vicente Díaz
Vicente.diaz5@mail.udp.cl
Isidora González
isidora.gonzalez4@mail.udp.cl
Natalia Ortega
natalia.ortega1@mail.udp.cl
Joaquín Villegas
joaquin.villegas@mail.udp.cl

Comunicación

- Componente clave en la distribución de los sistemas.
- Envío de mensajes a través de la red
- Problema: La comunicación se realiza a través de internet, sistemas poco seguros y saturados, es por esto que la red determina como se diseña la arquitectura del sistema.



Fundamentos de la red

Red:

- Variados dispositivos de hardware (cables, routers, switches, etc).
- Variados componentes de software.
- Medio físico para la comunicación.

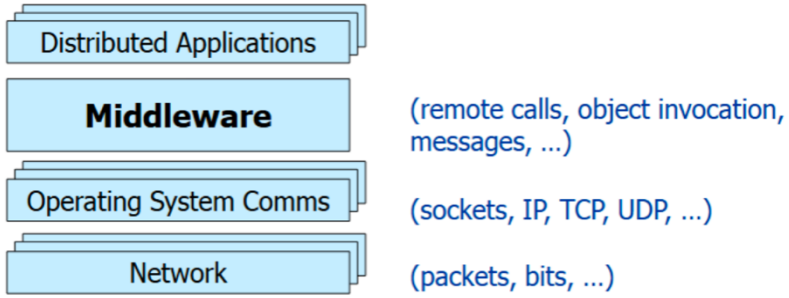
Performance:

- Latencia (tL): tiempo de transmitir un “mensaje vacío” (afectada por: sobrecarga de software, ruteo, carga de acceso al canal).
- Tasa de transferencia (tT o BW): velocidad de transmisión de datos (bit/seg) (afectada por características físicas de la red).
- Tiempo para transmitir un mensaje: M (T_m) de b bits: T_m = tL + b/tT
- Jitter: Variación del tiempo que toma entregar una serie de mensajes. Relevante en datos multimedia y gran parte de las aplicaciones actuales.

Fundamentos de la red

- **LAN (local area network):**
  - Alta velocidad. Distribuida en segmentos.
- **MAN (metropolitan area network):**
  - Alta velocidad en regiones más grande.
  - Útil para los requerimientos de los sistemas distribuidos (velocidad).
- **WAN (wide area network):**
  - Velocidad menor.
  - Conecta diversas organizaciones físicamente distantes.
  - Latencia depende de la ruta a seguir.
- **Wireless networks:**
  - Útiles para dispositivos móviles.
  - Alternativa a LAN cableadas.
- **Internetworks:**
  - Conexión de varios subsistemas de redes.
  - Uso de diferentes tecnologías y protocolos.
  - Útil para el desarrollo de sistemas distribuidos extensibles y abiertos.

Comunicación: Fundamentos



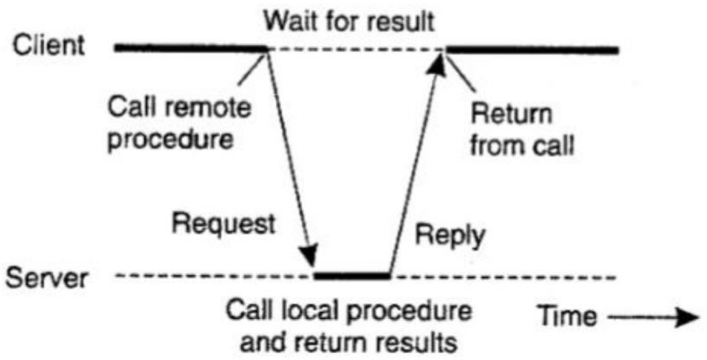
Protocolo

- Conjunto de reglas que controlan la comunicación entre entidades en una red
- Define codificación, inicio/fin del mensaje y detección de errores
- Estandariza representación de datos (enteros, strings, endianness)

Middleware

- Capa de software entre el OS y las aplicaciones distribuidas
- Ofrece servicios como llamadas remotas, mensajería e invocación de objetos
- Abstrae los detalles de red (sockets, IP, TCP, UDP)
- Ejemplos: sockets, RPC, RMI, pub/sub

¿Cómo se comunican los procesos?



Interprocess Comm.

- Comunicación directa de bajo nivel
- Usa primitivas send/receive
- Ejemplo: Sockets, MPI

Remote Invocation

- Comunicación bidireccional (request/reply)
- Paradigma más común en SD
- Ejemplo: RPC, RMI

Indirect Communication

- Tercera entidad como intermediario
- Desacople temporal y espacial
- Emisor y receptor no se conocen
- Ejemplo: Pub/Sub, Message Queues

Message Queues vs Pub/Sub

Message Queues

- Delegan trabajo de un servicio a otro
- Sin orden garantizado
- Si un consumidor cae, otro toma el mensaje
- Comunicación más durable que sincronía tradicional
- Ideal para escalar horizontalmente

Pub/Sub

- Todos los suscriptores reciben al menos 1 copia del mensaje
- Mismo orden para todos los suscriptores
- Cada suscriptor tiene su propia cola
- Al recuperarse, consume eventos pendientes de su cola
- Ideal para difundir eventos a múltiples receptores

RPC: Concepto y Propiedades

Protocolo que permite invocar un procedimiento en otra máquina sin conocer los detalles de red — modelo Cliente/Servidor

Propiedades

- Nivel de lenguaje: fácil para el programador
- Request/reply síncrono
- Distribución transparente al programador
- Garantía parcial ante fallas (retry)

Flujo básico

- Proceso A invoca procedimiento en máquina B
- Define parámetros y espera resultado
- El cliente se bloquea durante la llamada
- El servidor ejecuta y retorna el resultado

RPC: Desventajas y Fallas

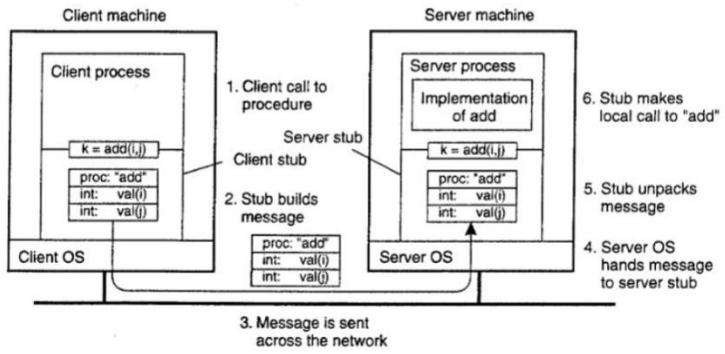
Desventajas

- Atado al modelo Cliente/Servidor
- Síncrono: cliente bloqueado esperando
- Overhead por marshalling/unmarshalling
- No hay transparencia total
- No es orientado a objetos (se invocan funciones, no métodos)

Tipos de Fallas

- Congestión de red o servidor
- Caída del cliente durante la llamada
- Caída del servidor durante la ejecución
- Falla de red entre cliente y servidor
- Manejo: el error se delega al programador (retry)

Stub Cliente / Servidor



Stub Cliente

- Objetivo: hacer la llamada remota igual a una local
- El llamador no nota que se ejecuta en otra máquina
- Empaqueta (marshalling) los parámetros
- Los envía por la red en vez de solicitarlos al SO
- Al recibir respuesta, desempaqueta el resultado

Stub Servidor

- Recibe el mensaje y lo transforma a llamadas locales
- Se bloquea esperando la invocación del cliente
- Desempaqueta los parámetros (unmarshalling)
- Llama al procedimiento local con los parámetros
- Retorna el resultado al stub cliente

El paso de mensajes queda oculto — no hay invocación explícita a send/receive

Paso de Parámetros y Java RMI

Marshalling: transformación de datos al formato adecuado para transmisión en la red

Por Valor

- Copia del dato se envía en el mensaje
- Representación de datos (ASCII, EBCDIC)
- Problema de endianness (little/big endian)

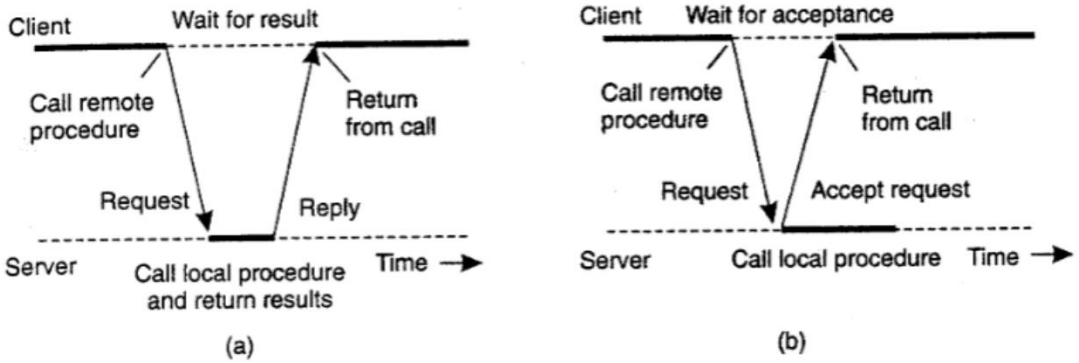
Por Referencia

- Las referencias solo tienen sentido localmente
- Solución: copiar el arreglo completo y enviarlo
- Servidor lo referencia en su espacio local
- El arreglo modificado se retorna al cliente

Java RMI

- RPC orientado a objetos en Java
- Objetos remotos con interfaces remotas
- Stub en cliente, Skeleton en servidor
- Enmascara objetos remotos con un proxy

RPC Asíncrono



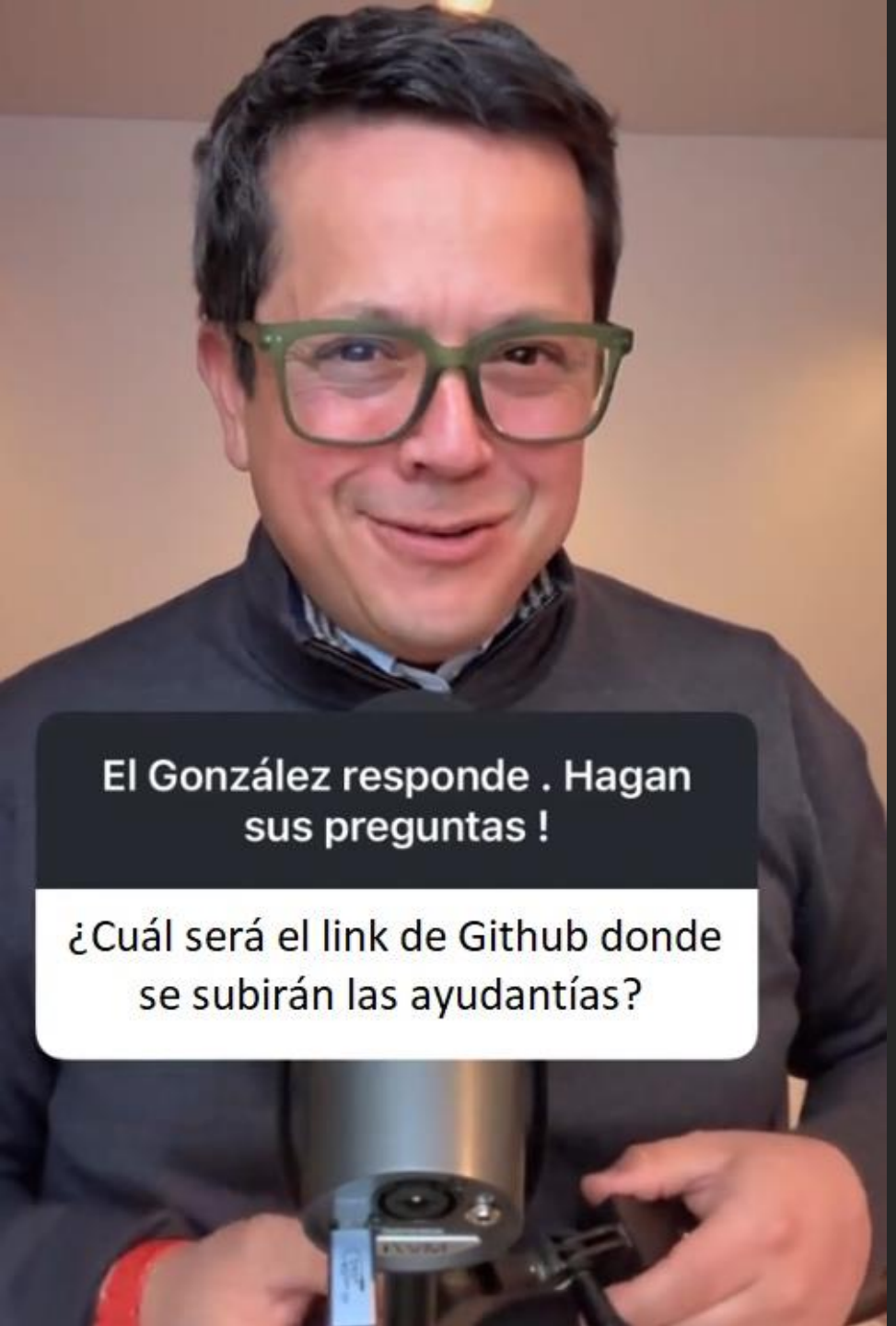
Por defecto, el RPC es síncrono y bloqueante: el cliente espera hasta recibir el resultado completo

RPC Síncrono (clásico)

- Cliente envía request al servidor
- Cliente queda bloqueado esperando reply
- Solo continúa al recibir el resultado completo
- Sencillo de implementar pero ineficiente

RPC Asíncrono (optimizado)

- Servidor envía ACK de aceptación inmediata
- Cliente no queda bloqueado, continúa ejecutando
- Implementado con multi-threading
- El resultado se recibe en otro momento
- Mayor rendimiento y escalabilidad



Link Github

Repositorio:

https://github.com/VicenteDiazH/AY-SD2026-1-S3